



Computational Physics @ NuSTEAM

Lecture 2

- *Numerical integration*
- *Numerical differentiation*

June 12, 2025

Instructor: Volodymyr Vovchenko (vvovchenko@uh.edu)

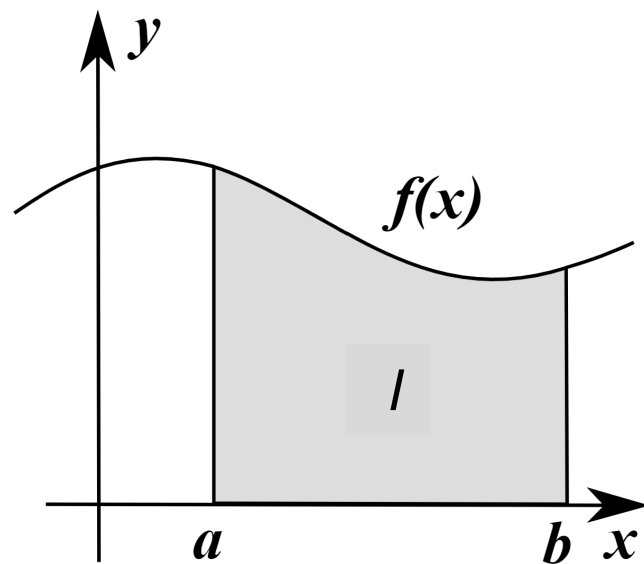
based on Computations Physics course at UH

GitHub: <https://github.com/vlvovch/PHYS6350-ComputationalPhysics/tree/nusteam>

Online textbook: <https://vovchenko.net/computational-physics>

Numerical integration

$$I = \int_a^b f(x) dx$$



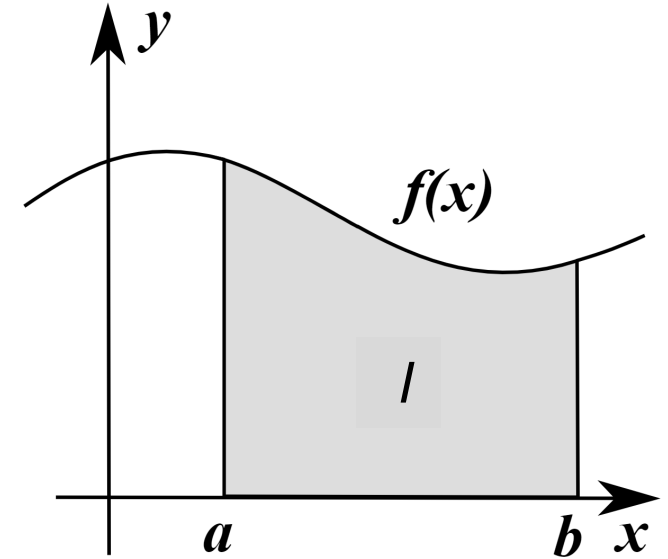
Numerical integration

Generic problem: evaluate

$$I = \int_a^b f(x) dx$$

We need numerical integration when

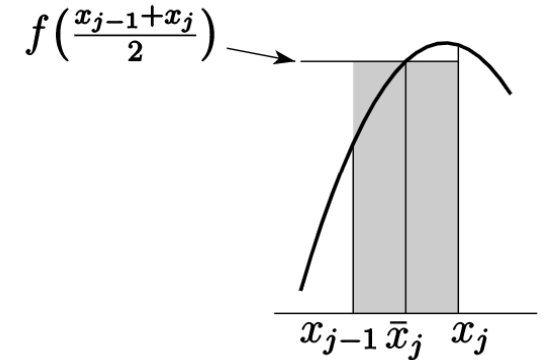
- Cannot/difficult integrate analytically
- Only know the integrand $f(x)$ at certain points x



Numerical integration: rectangle (midpoint) rule

Interpret the integral as the area under the curve and approximate by a rectangle evaluated at midpoint

$$\int_a^b f(x) dx \approx (b - a) f\left(\frac{a + b}{2}\right)$$



Error (from Euler-McLaurin formula):

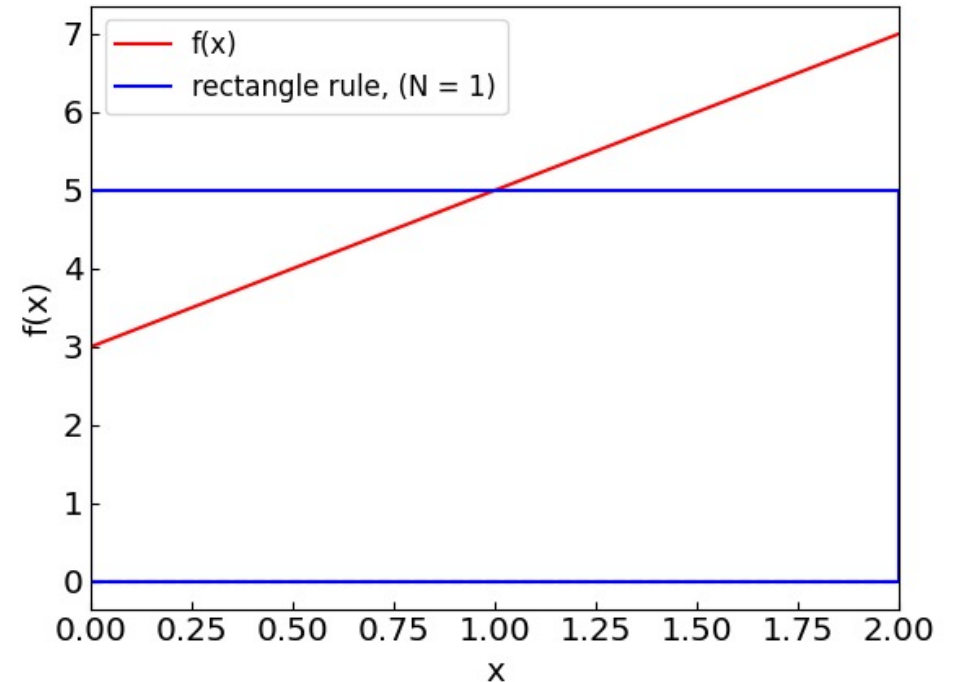
$$\int_a^b f(x) dx - (b - a) f\left(\frac{a + b}{2}\right) \approx \frac{(b - a)^3}{24} f''(a)$$

The rule is exact for the integration of linear functions

Numerical integration: rectangle (midpoint) rule

Example:

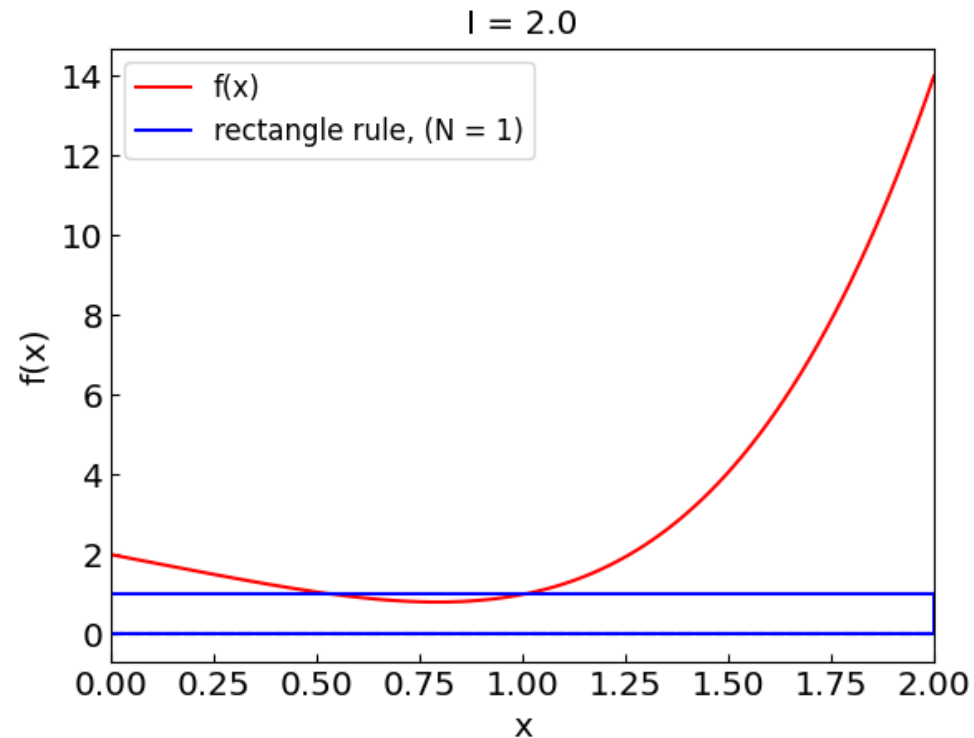
$$I = \int_0^2 (2x + 3)dx = 10$$



Although the rectangle is a poor approximate of the line (which is a trapezoid here), the errors cancel out

Numerical integration: rectangle (midpoint) rule

Another example: $I = \int_0^2 (x^4 - 2x + 2)dx = 6.4$



Rectangle rule gives $I_{\text{rect}} = 2$ which is way off

Extended (composite) rectangle rule

Split the integration interval into N sub-intervals and apply the rectangle rule separately to each one

$$\int_a^b f(x) \approx h \sum_{k=1}^N f(x_k), \quad k = 1, \dots, N$$

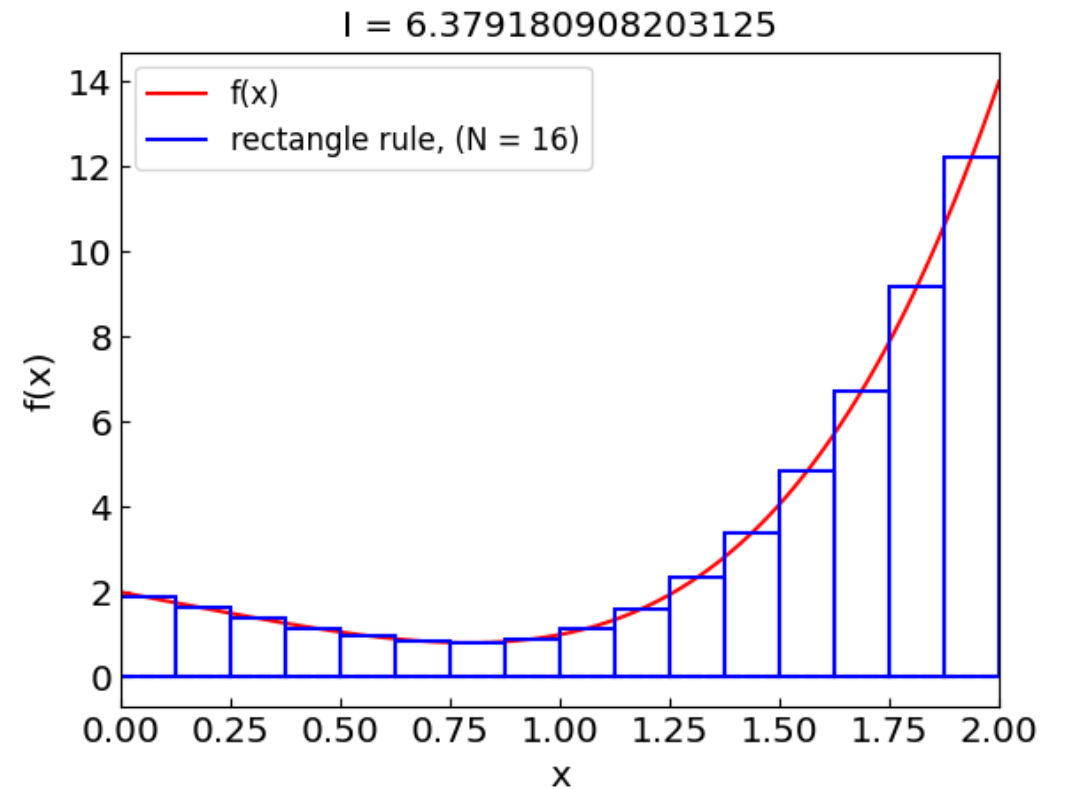
$$x_k = a + \frac{2k-1}{2}h.$$

$$h = (b-a)/N$$

Error estimate:

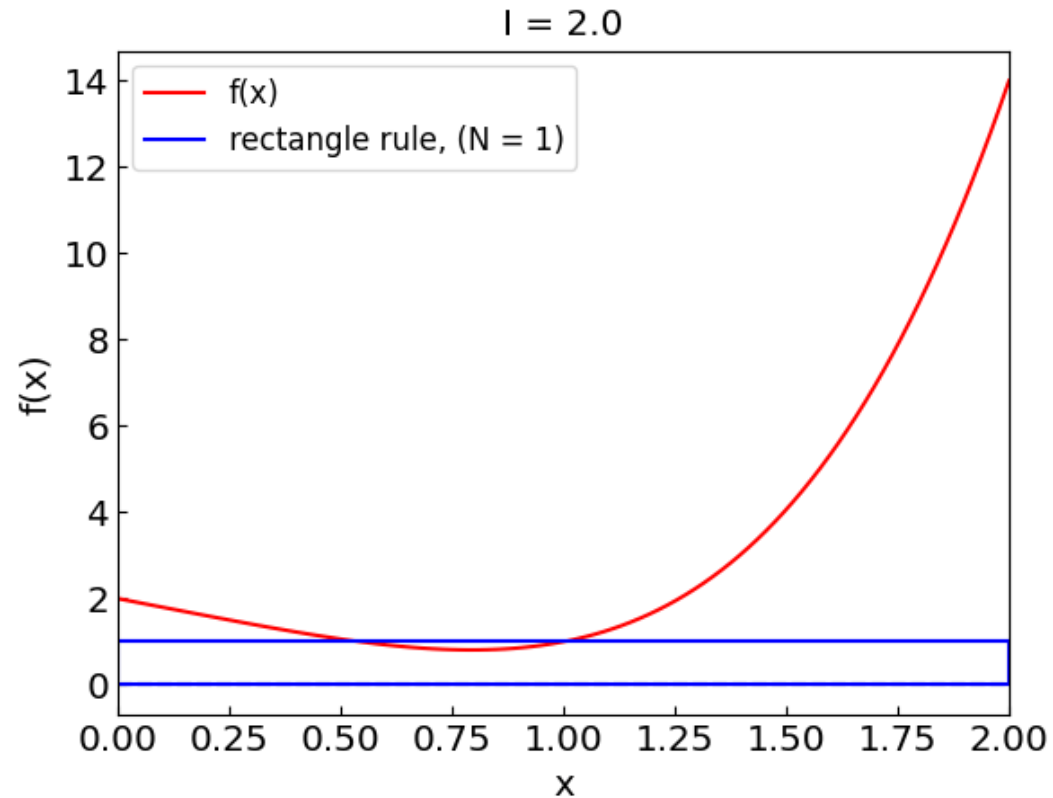
$$I - I_{\text{rect}} = (b-a) \frac{h^2}{24} f''(a) + \mathcal{O}(h^4)$$

$$I = \int_0^2 (x^4 - 2x + 2) dx = 6.4$$



Extended (composite) rectangle rule

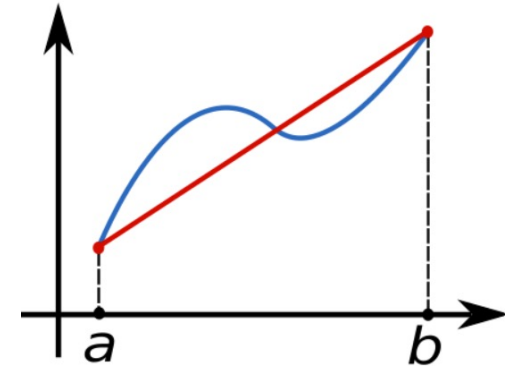
$$I = \int_0^2 (x^4 - 2x + 2) dx = 6.4$$



Numerical integration: trapezoidal rule

Approximate the integral by a trapezoid

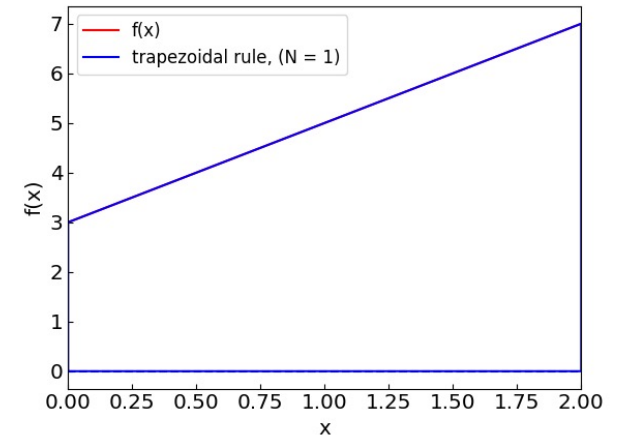
$$\int_a^b f(x) dx \approx (b - a) \frac{f(a) + f(b)}{2}$$



Error:

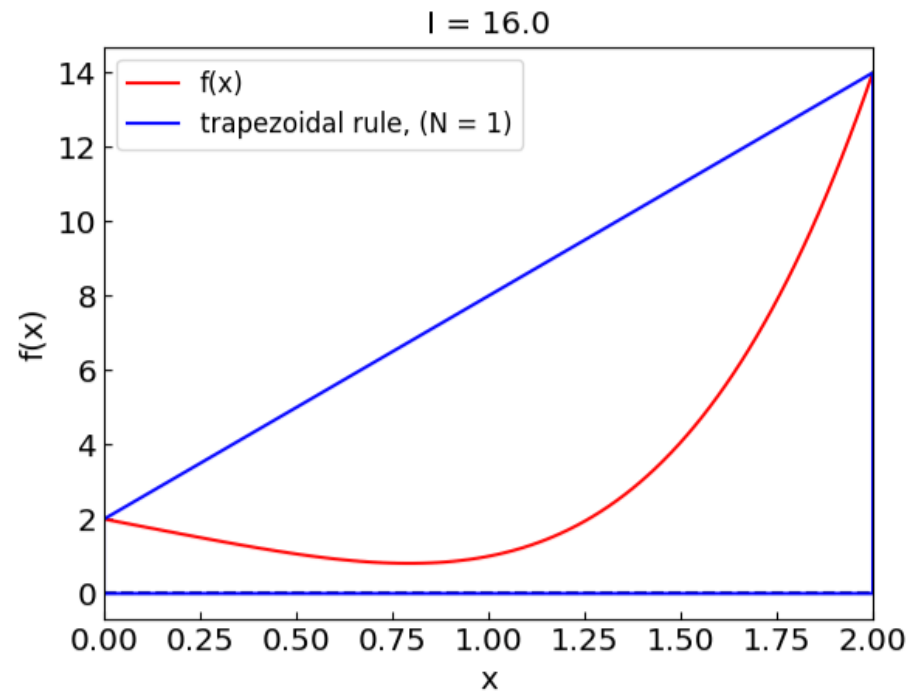
$$\int_a^b f(x) dx - (b - a) \frac{f(a) + f(b)}{2} \approx -\frac{(b - a)^3}{12} f''(a)$$

The rule is exact for the integration of linear functions



Numerical integration: trapezoidal rule

$$I = \int_0^2 (x^4 - 2x + 2) dx = 6.4$$



Trapezoidal rule gives $I_{\text{trap}} = 16$, way off and in the opposite direction relative to rectangle rule

Extended trapezoidal rule

$$\int_0^2 x^4 - 2x + 2$$

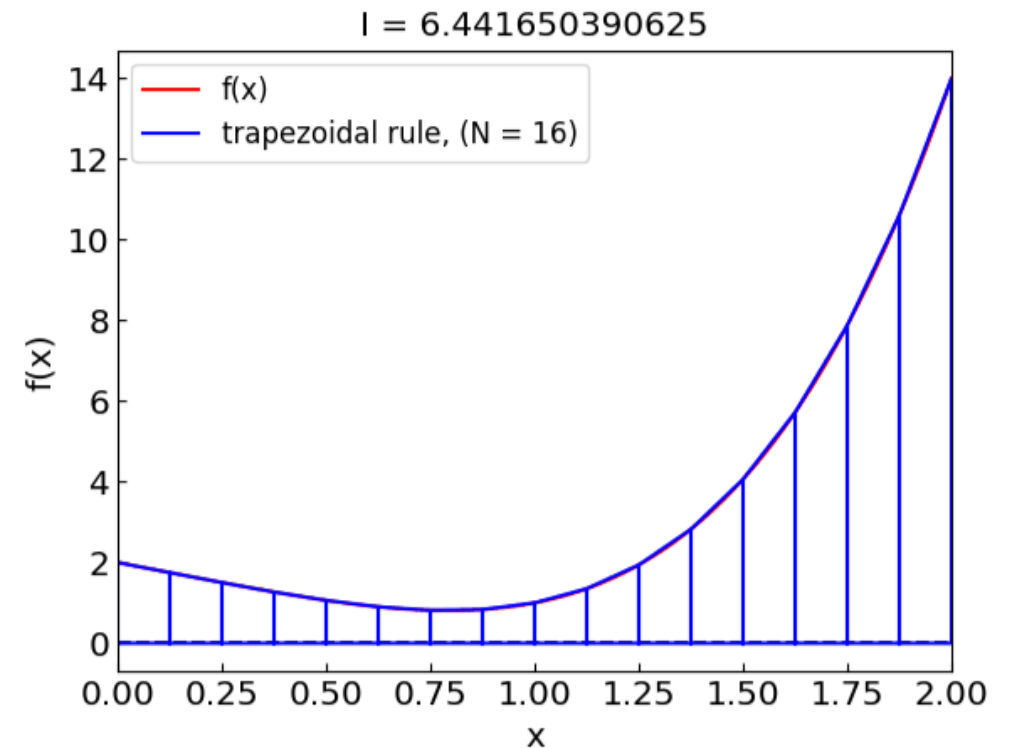
$$\int_a^b f(x) \approx h \sum_{k=0}^N \frac{f(x_k) + f(x_{k+1})}{2}, \quad i = 0, \dots, N$$

$$x_k = a + kh.$$

$$h = (b - a)/N$$

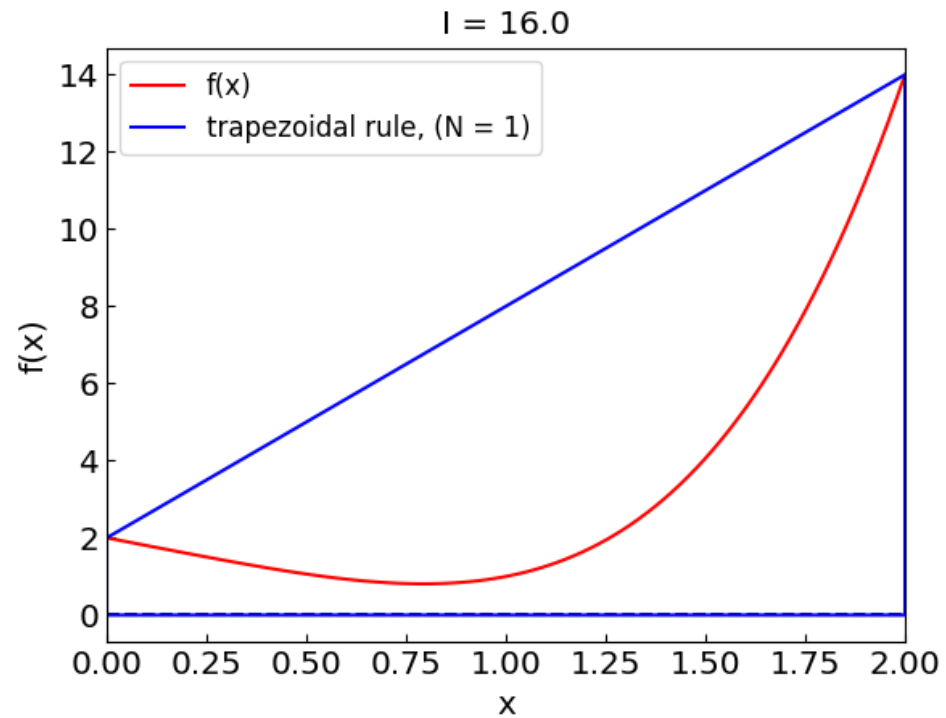
Error estimate:

$$I - I_{\text{trap}} = -(b - a) \frac{h^2}{12} f''(a) + \mathcal{O}(h^4)$$



Extended (composite) trapezoidal rule

$$I = \int_0^2 x^4 - 2x + 2 = 6.4$$



Numerical integration: Simpson's rule

Recall the error estimates for rectangular and trapezoidal rules

$$I - I_{\text{rect}} = (b - a) \frac{h^2}{24} f''(a) + \mathcal{O}(h^4) \qquad I - I_{\text{trap}} = -(b - a) \frac{h^2}{12} f''(a) + \mathcal{O}(h^4)$$

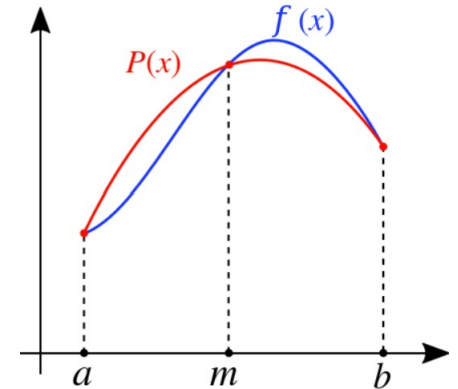
Combine them to eliminate the $\mathcal{O}(h^2)$ error term:

$$I_S = \frac{2I_{\text{rect}} + I_{\text{trap}}}{3}$$

i.e.

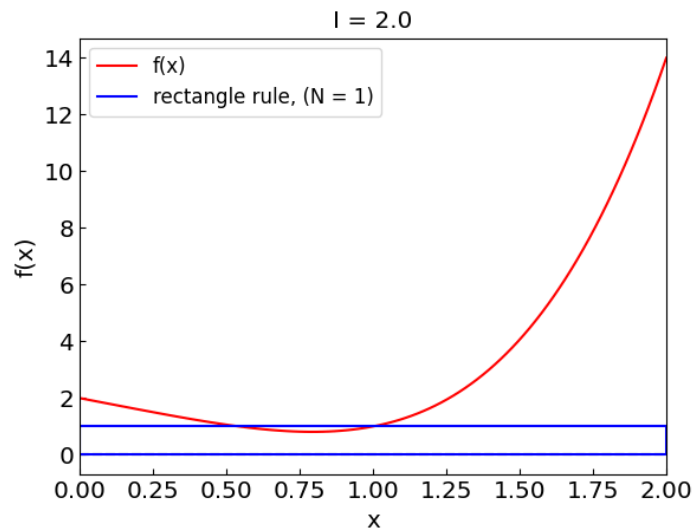
$$\int_a^b f(x) dx \approx \frac{(b - a)}{6} \left[f(a) + 4f\left(\frac{a + b}{2}\right) + f(b) \right]$$

An equivalent way to obtain the rule: replace the integrand by the parabolic interpolation

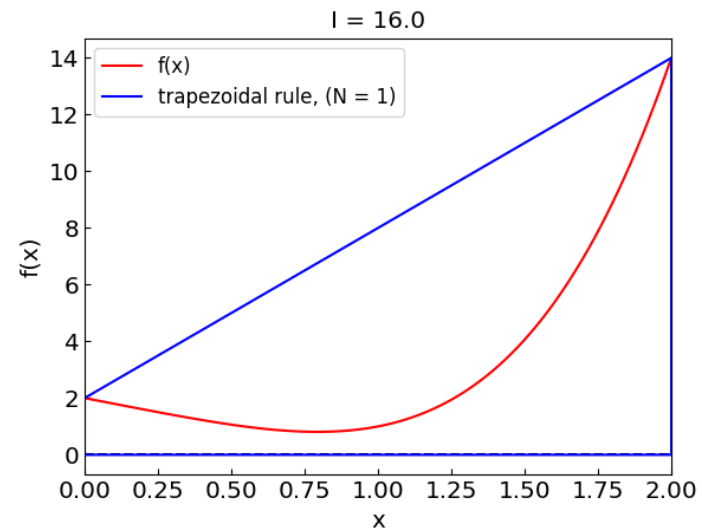


Comparing the methods

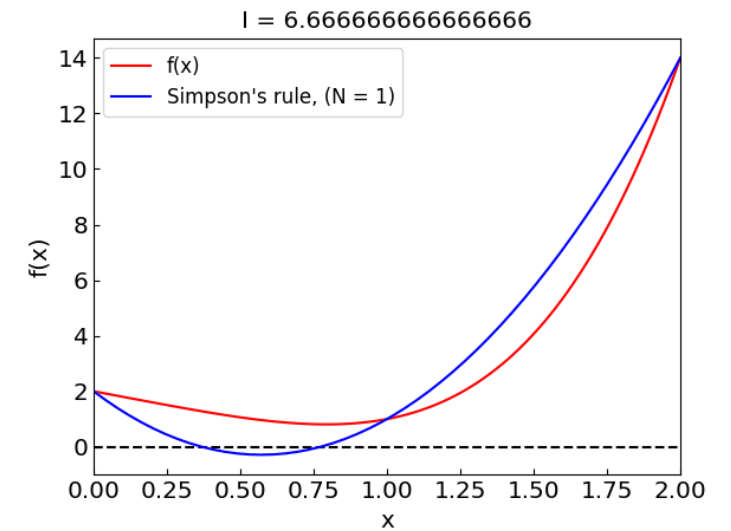
Rectangle



Trapezoid



Simpson



Advanced methods

- Adaptive quadrature
 - Increase (e.g. double) the number of subintervals and use the difference as the measure of error, e.g.
 - More formal implementation: **Romberg method**
- **Gaussian quadrature**
 - N-point quadrature which is exact for integration of polynomials of order up to $2n-1$

Adaptive trapezoidal rule

$$\varepsilon_N \simeq (I_{\text{trap}}^{2N} - I_{\text{trap}}^N)/3$$

$$\int_a^b f(x) dx \approx \sum_k w_k f(x_k)$$

- Improper integrals
 - (Semi-)infinite integration limits
 - Singularities
 - Discontinuous integrals

Improper integrals

- Contain integrable singularities (typically at the endpoints)

$$\int_0^1 \frac{1}{\sqrt{x}} dx = 2\sqrt{x} \Big|_0^1 = 2$$

- (Semi-)infinite integration range

$$\int_0^\infty e^{-x} dx = 1$$

$$\int_{-\infty}^\infty e^{-x^2} dx = \sqrt{\pi}$$

Example: Momentum integration over thermal distributions (Fermi-Dirac/Bose-Einstein)

$$n = \frac{d}{2\pi^2} \int_0^\infty dk k^2 \left[\exp \left\{ \frac{\sqrt{m^2 + k^2} - \mu}{T} \right\} \pm 1 \right]^{-1}$$

Improper integrals: (Semi-)infinite intervals

Solution: map to a finite interval [e.g. (0,1)] by a change of variables

- Semi-infinite:

$$\int_a^\infty f(x)dx \quad \longrightarrow \quad x = a + \frac{t}{1-t} \quad \longrightarrow \quad \int_a^\infty f(x)dx = \int_0^1 f\left(a + \frac{t}{1-t}\right) \frac{dt}{1-t^2} = \int_0^1 g(t)dt$$

- Infinite:

$$\int_{-\infty}^\infty f(x)dx \quad \longrightarrow \quad x = \frac{t}{1-t^2} \quad \longrightarrow \quad \int_{-\infty}^\infty f(x)dx = \int_{-1}^1 f\left(\frac{t}{1-t^2}\right) \frac{1+t^2}{(1-t^2)^2} dt = \int_{-1}^1 g(t)dt$$

Then apply a standard method (e.g. rectangle rule to avoid endpoint singularities) to $g(t)$

NB: Other options for the change of variable are possible

Improper integrals: Semi-infinite intervals

$$\int_0^{\infty} e^{-x} dx = 1$$

```
def fexp(x):  
    return np.exp(-x)  
  
def g(t, f, a = 0.):  
    return f(a + t / (1. - t)) / (1. - t)**2
```

```
a = 0.  
def frect(x):  
    return g(x, fexp, a)  
  
print('Using change of variable and the rectangle rule to evaluate \int_0^\infty \exp(-x) dx')  
rectangle_rule_adaptive(frect, 0., 1., 1, 1.e-6, 20)
```

Using change of variable and the rectangle rule to evaluate $\int_0^\infty \exp(-x) dx$

```
Iteration:    1, I =    1.471517764685769  
Iteration:    2, I =    1.035213267452946, error estimate = -0.145434832410941  
Iteration:    3, I =    0.984670579385046, error estimate = -0.016847562689300  
Iteration:    4, I =    1.001784913275257, error estimate = 0.005704777963404  
Iteration:    5, I =    1.000155714391028, error estimate = -0.000543066294743  
Iteration:    6, I =    1.000040642390661, error estimate = -0.000038357333456  
Iteration:    7, I =    1.000010172618432, error estimate = -0.000010156590743  
Iteration:    8, I =    1.000002543136036, error estimate = -0.000002543160799  
Iteration:    9, I =    1.000000635783161, error estimate = -0.000000635784292
```

Improper integrals: Infinite intervals

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi} = 1.772454 \dots$$

```
def fexp2(x):  
    return np.exp(-x**2)  
  
def g2(t, f):  
    return f(t / (1. - t**2)) * (1.+t**2) / (1. - t**2)**2  
  
def frect2(x):  
    return g2(x, fexp2)  
  
print('Using change of variable and the rectangle rule to evaluate \int_{-\infty}^{\infty} \exp(-x^2) dx')  
rectangle_rule_adaptive(frect2, -1., 1., 1, 1.e-6, 20)  
  
print('Expected value: \sqrt{\pi} =', np.sqrt(np.pi))
```

```
Using change of variable and the rectangle rule to evaluate \int_{-\infty}^{\infty} \exp(-x^2) dx  
Iteration:    1, I =      2.000000000000000  
Iteration:    2, I =      2.849690615244243, error estimate = 0.283230205081414  
Iteration:    3, I =      1.557994553948652, error estimate = -0.430565353765197  
Iteration:    4, I =      1.808005109208286, error estimate = 0.083336851753211  
Iteration:    5, I =      1.770118560572371, error estimate = -0.012628849545305  
Iteration:    6, I =      1.772492101507391, error estimate = 0.000791180311673  
Iteration:    7, I =      1.772453880915058, error estimate = -0.000012740197444  
Iteration:    8, I =      1.772453850905505, error estimate = -0.000000010003185  
Expected value: \sqrt{\pi} = 1.7724538509055159
```

Summary: Numerical integration

- Trapezoidal/rectangle method
 - Good generic choice
- Gaussian quadrature
 - Good choice for repeatedly calculating the same kind of integral
- Improper integrals
 - Make a change of variable to transform into finite interval
 - *Advanced:* Using generalized Gaussian quadratures
- Control over error
 - *Advanced:* Romberg method

Numerical derivatives

$$\frac{df}{dx} \simeq \frac{f(x+h) - f(x)}{h}$$

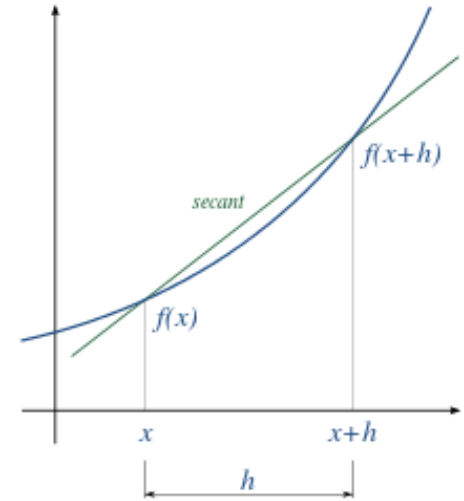
Numerical differentiation

Generic problem: evaluate

$$\frac{df}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

We need numerical differentiation when

- Function f is known at a discrete set of points
- Too expensive/cumbersome to do directly
 - E.g. when $f(x)$ itself is a solution to a complex web of non-linear equations, calculating $f'(x)$ explicitly will require rewriting all the equations



Forward difference

Simply approximate

$$\frac{df}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

by

$$\frac{df}{dx} \simeq \frac{f(x+h) - f(x)}{h}$$

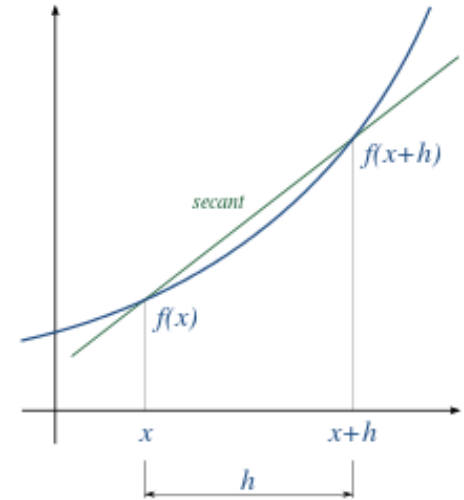
where h is finite

Taylor theorem:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \dots$$

gives the approximation error estimate of

$$R_{\text{forw}} = -\frac{1}{2}hf''(x) + \mathcal{O}(h^2)$$



Backward difference

Backward difference

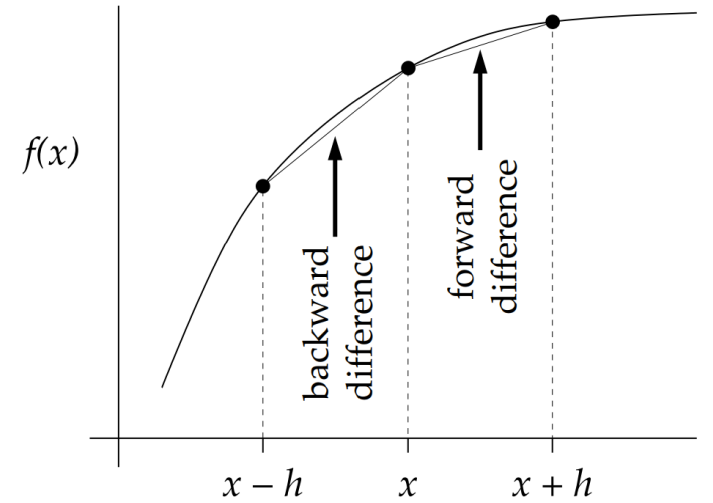
$$\frac{df}{dx} \simeq \frac{f(x) - f(x-h)}{h}$$

Taylor theorem:

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) + \dots$$

gives the approximation error estimate of

$$R_{\text{back}} = \frac{1}{2}hf''(x) + \mathcal{O}(h^2)$$



Central difference

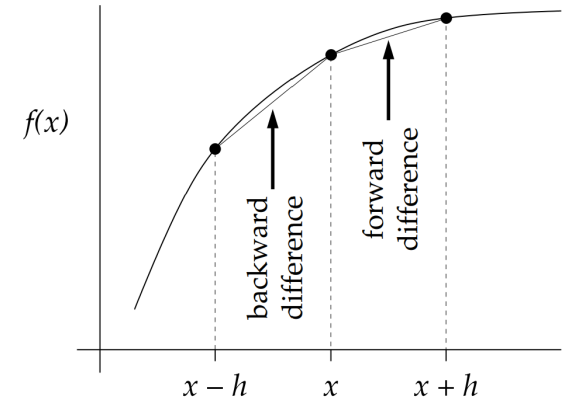
Recall the forward and backward difference and their errors

$$\frac{df}{dx} \simeq \frac{f(x+h) - f(x)}{h}$$

$$R_{\text{forw}} = -\frac{1}{2}hf''(x) + \mathcal{O}(h^2)$$

$$\frac{df}{dx} \simeq \frac{f(x) - f(x-h)}{h}$$

$$R_{\text{back}} = \frac{1}{2}hf''(x) + \mathcal{O}(h^2)$$



Taking the average of the two cancels out the $\mathcal{O}(h)$ error term

central difference $\frac{df}{dx} \simeq \frac{f(x+h) - f(x-h)}{2h}$

Error estimate:

$$R_{\text{cent}} = -\frac{f'''(x)}{6}h^2 + \mathcal{O}(h^3)$$

High-order central difference

To improve the approximation error use more than two function evaluations, e.g.

$$\frac{df}{dx} \simeq \frac{Af(x+2h) + Bf(x+h) + Cf(x) + Df(x-h) + Ef(x-2h)}{h} + O(h^4)$$

Determine A, B, C, D, E using Taylor expansion to cancel all terms up to h^4

$$\frac{df}{dx} \simeq \frac{-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)}{12h} + \frac{h^4}{30} f^{(5)}(x)$$

High-order terms:

Derivative	Accuracy	-5	-4	-3	-2	-1	0	1	2	3	4	5
1	2					-1/2	0	1/2				
	4				1/12	-2/3	0	2/3	-1/12			
	6			-1/60	3/20	-3/4	0	3/4	-3/20	1/60		
	8		1/280	-4/105	1/5	-4/5	0	4/5	-1/5	4/105	-1/280	

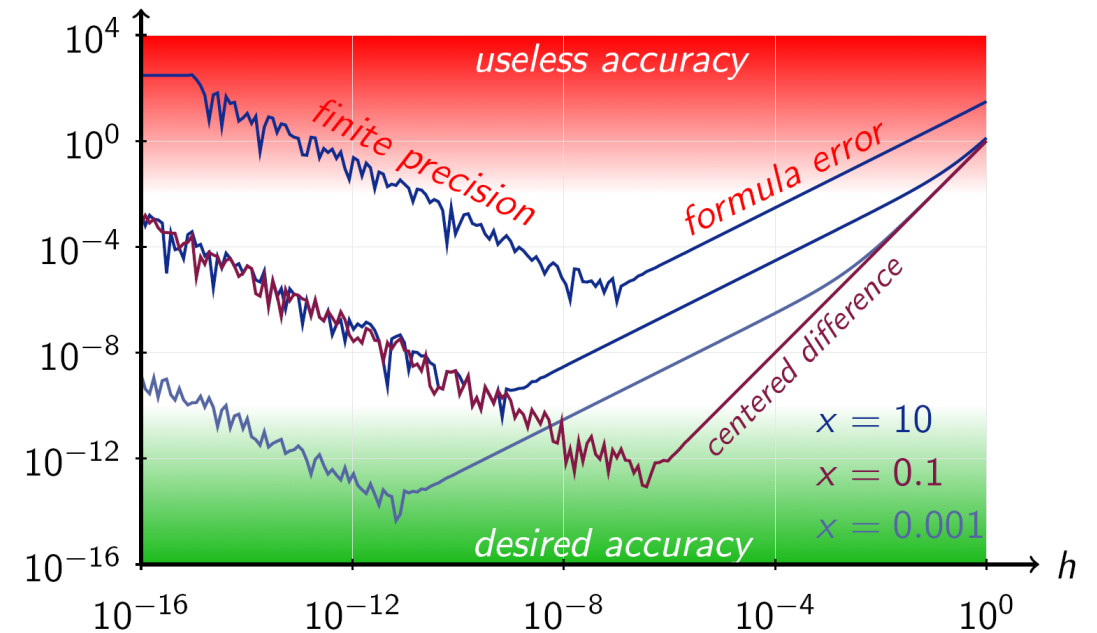
Balancing truncation and round-off errors

If h is too small, round-off errors become important

- cannot distinguish x and $x+h$ and/or $f(x+h)$ and $f(x)$ with enough accuracy

As a rule of thumb, if ε is machine precision and the truncation error is of order $O(h^n)$, then h should not be much smaller than $h \sim \sqrt[n+1]{\varepsilon}$

The higher the finite difference order is, the larger h should be



Credit: Wikipedia

Balancing truncation and round-off errors

Consider central difference:

$$\frac{df}{dx} \simeq \frac{f(x+h) - f(x-h)}{2h}$$

Total error:

$$\text{error}(df/dx) = \underbrace{\varepsilon_m \frac{|f(x)|}{h}}_{\text{round-off}} + \underbrace{\frac{|f'''(x)|}{6} h^2}_{\text{truncation}}$$

ε_m - relative error in $f(x)$

$$\varepsilon_m \sim 10^{-16}$$

Best case: machine precision

Minimizing with respect to h gives optimal choice for the step size:

$$h = \sqrt[3]{6 \varepsilon_m |f'''(x)| / |f(x)|} \sim \sqrt[3]{\varepsilon_m |f'''(x)| / |f(x)|}$$

More generally, for $O(h^n)$ scheme one has

$$h \sim \sqrt[n+1]{\varepsilon_m |f^{(n+1)}(x)| / |f(x)|}$$

Balancing truncation and round-off errors

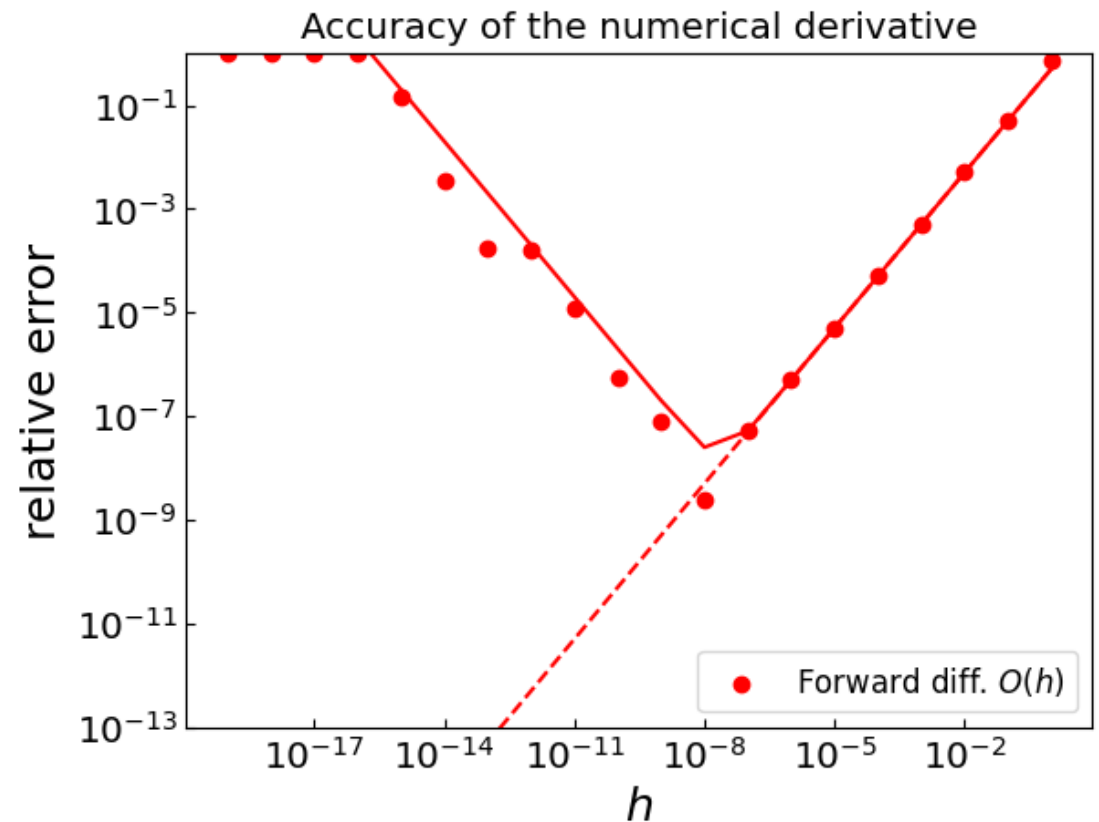
Let $f(x) = \exp(x)$

Calculate the derivatives at $x = 0$

```
def f(x):  
    return np.exp(x)  
  
def df(x):  
    return np.exp(x)
```

Forward difference $O(h)$:

Optimal $h \sim \sqrt[2]{10^{-16}} \sim 10^{-8}$



Balancing truncation and round-off errors

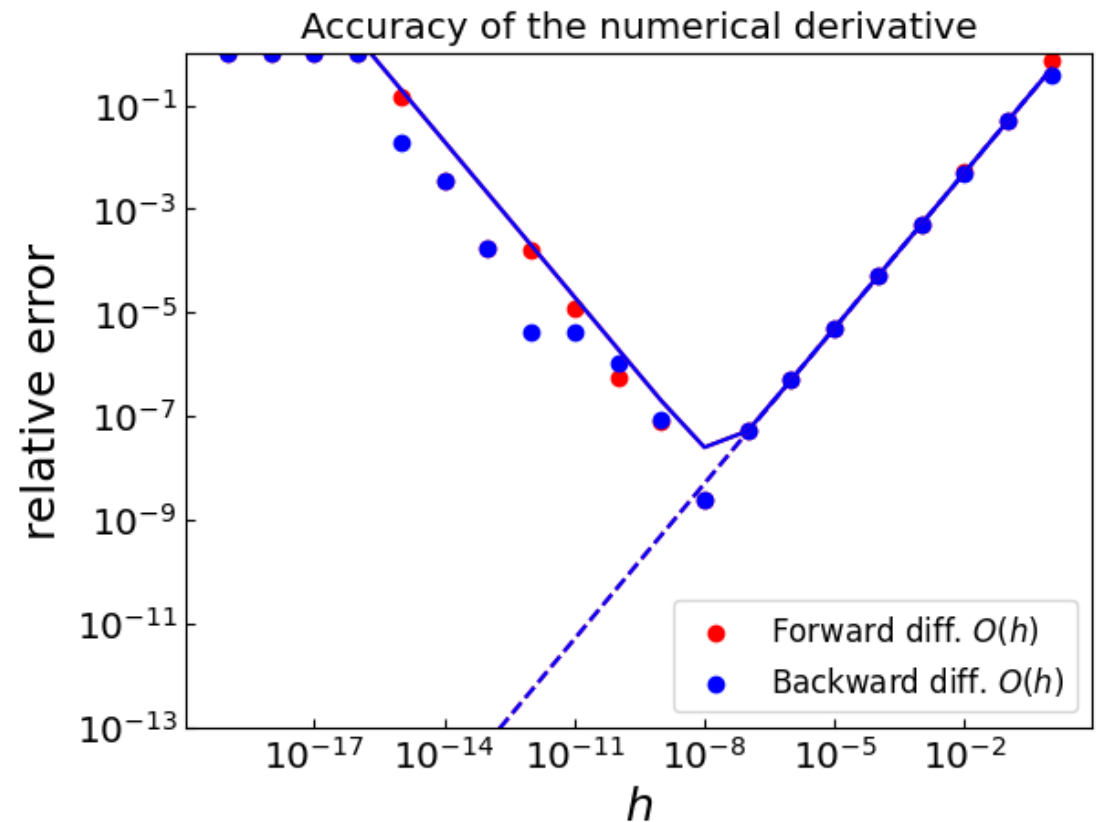
Let $f(x) = \exp(x)$

Calculate the derivatives at $x = 0$

```
def f(x):  
    return np.exp(x)  
  
def df(x):  
    return np.exp(x)
```

Backward difference $O(h)$:

Optimal $h \sim \sqrt[2]{10^{-16}} \sim 10^{-8}$



Balancing truncation and round-off errors

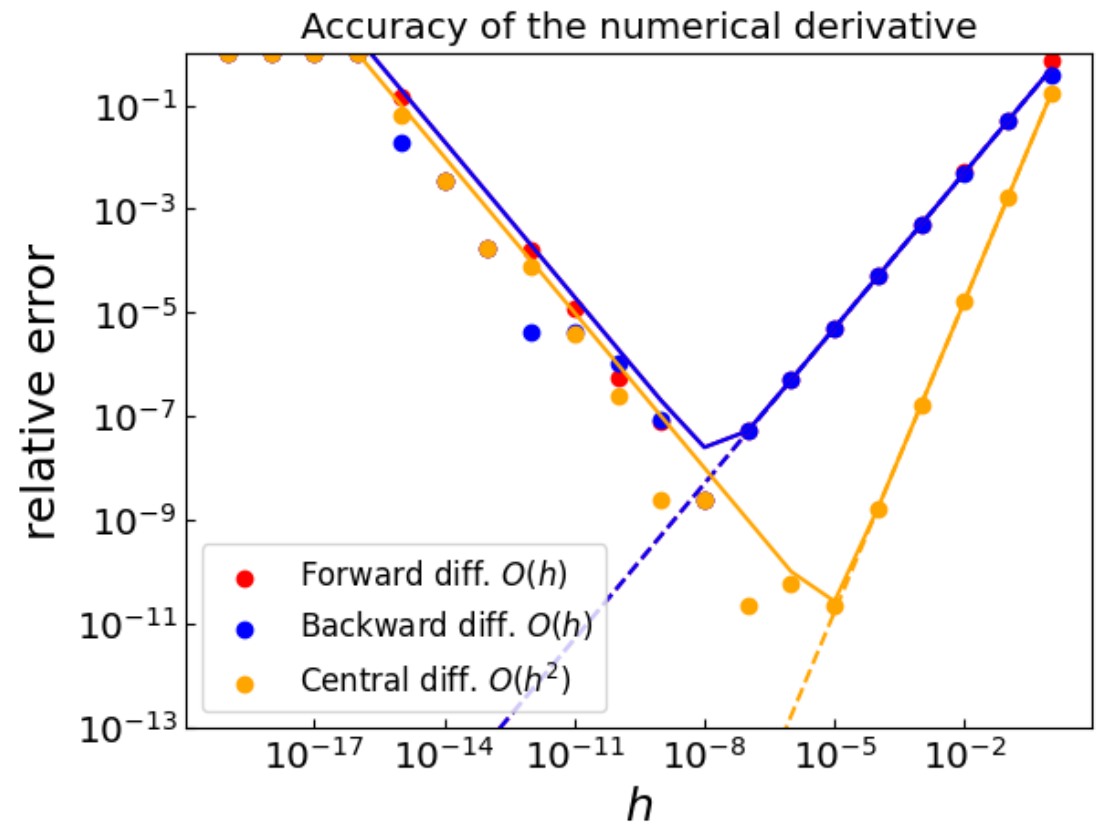
Let $f(x) = \exp(x)$

Calculate the derivatives at $x = 0$

```
def f(x):  
    return np.exp(x)  
  
def df(x):  
    return np.exp(x)
```

Central difference $O(h^2)$:

Optimal $h \sim \sqrt[3]{10^{-16}} \sim 10^{-5}$



Balancing truncation and round-off errors

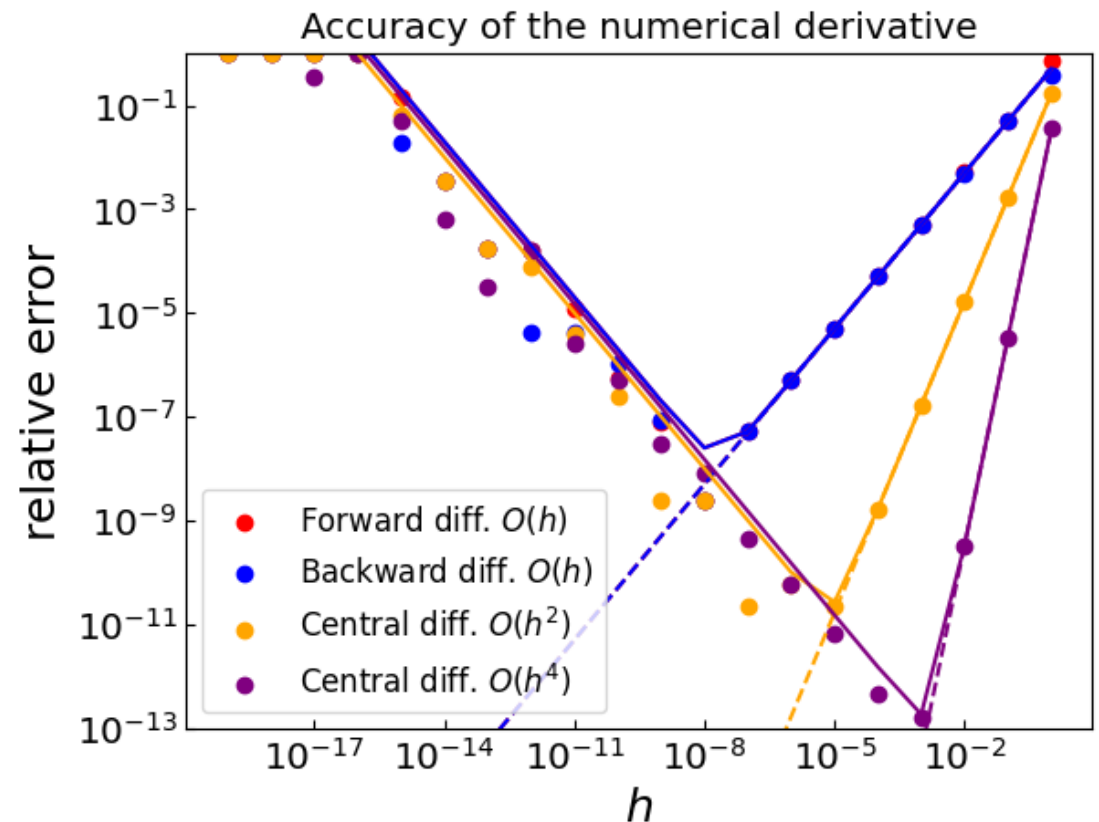
Let $f(x) = \exp(x)$

Calculate the derivatives at $x = 0$

```
def f(x):  
    return np.exp(x)  
  
def df(x):  
    return np.exp(x)
```

Central difference $O(h^4)$:

Optimal $h \sim \sqrt[5]{10^{-16}} \sim 10^{-3}$



High-order derivatives

Central difference

$$\frac{df}{dx}(x) \simeq \frac{f(x + h/2) - f(x - h/2)}{h}$$

Now apply the central difference again to $f'(x+h/2)$ and $f'(x-h/2)$

$$\begin{aligned} f''(x) &\simeq \frac{f'(x + h/2) - f'(x - h/2)}{h} \\ &= \frac{[f(x + h) - f(x)]/h - [f(x) - f(x - h)]/h}{h} \\ &= \frac{f(x + h) - 2f(x) + f(x - h)}{h^2}. \end{aligned}$$

General formula [to order $O(h^2)$]

$$f^{(n)}(x) = \frac{1}{h^n} \sum_{k=0}^n (-1)^k \binom{n}{k} f[x + (n/2 - k)h] + O(h^2)$$

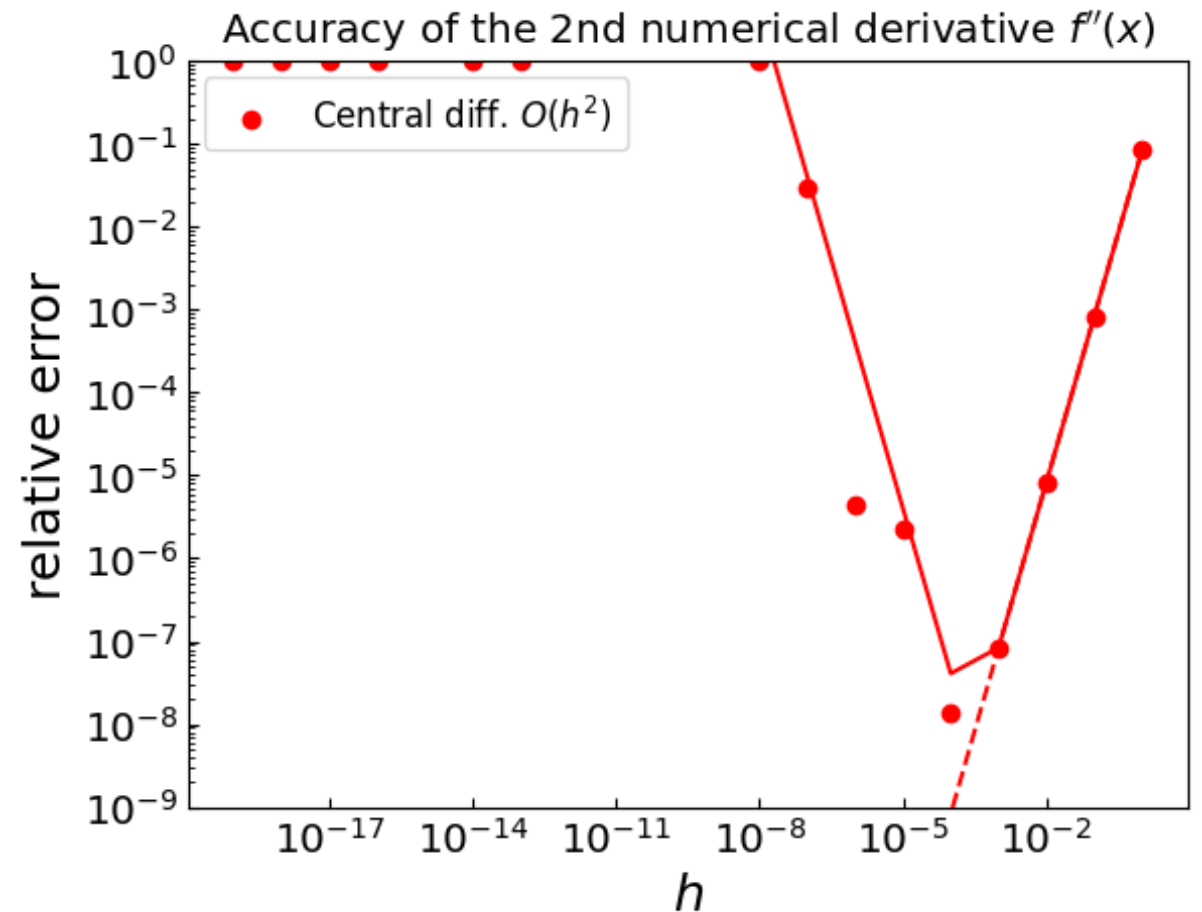
Second derivative

```
def d2f_central(f,x,h):  
    return (f(x+h) - 2*f(x) + f(x-h)) / (h**2)
```

$$f(x) = \exp(x)$$

```
def f(x):  
    return np.exp(x)  
  
def df(x):  
    return np.exp(x)  
  
def d2f(x):  
    return np.exp(x)
```

Optimal $h \sim \sqrt[4]{10^{-16}} \sim 10^{-4}$



Partial derivatives

Let us have a function of two variables: $f(x,y)$

Use central difference to calculate first-order derivatives

$$\frac{\partial f}{\partial x} = \frac{f(x + h/2, y) - f(x - h/2, y)}{h}$$
$$\frac{\partial f}{\partial y} = \frac{f(x, y + h/2) - f(x, y - h/2)}{h}$$

Reapply the central difference to calculate $\partial^2 f(x, y) / \partial x \partial y$

$$\frac{\partial^2 f}{\partial x \partial y} = \frac{f(x + h/2, y + h/2) - f(x - h/2, y + h/2) - f(x + h/2, y - h/2) + f(x - h/2, y - h/2)}{h^2}$$

Summary: Numerical differentiation

- **Forward/backward differences**
 - Useful when we are given a grid of function values
 - Need $f'(x)$ at the same point as x
 - Have limited accuracy (error is linear in h)
- **Central difference**
 - More precise than forward/backward differences (error is quadratic in h)
 - Gives $f'(x)$ estimate at the midpoint of function evaluation points
- **Higher-order formulas are obtained by using more than two function evaluations**
 - Can be used when limited number of function evaluations available
- **Straightforwardly extendable to high-order and partial derivatives**
- **Balance between truncation and round-off error must be respected**
 - h should not be taken too small

Advanced topic: Automatic differentiation ([link](#))

Extra slides

Adaptive quadrature

We would like to control the error in our calculation

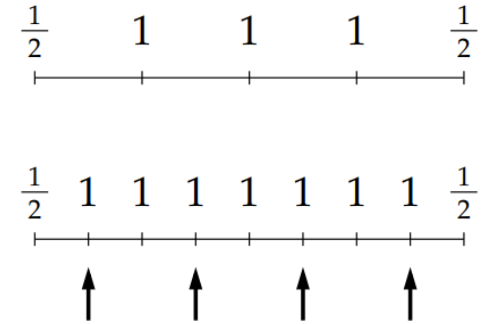
This can be achieved by doubling the number of subintervals and keeping track of the error estimate

Recall that in the rectangle/trapezoidal rule the error is proportional to h^2

$$I - I_{\text{trap}} \approx ch^2$$

At step k we have $h_k = h_{k-1}/2$ therefore $I - I_{\text{trap}}^k \approx ch_k^2$, $I - I_{\text{trap}}^{k-1} \approx 4ch_k^2$,
and the error at step k is estimated as

$$\varepsilon_k \simeq (I_{\text{trap}}^k - I_{\text{trap}}^{k-1})/3$$



Adaptive trapezoidal rule

Trapezoidal rule for numerical integration with adaptive step

```
def trapezoidal_rule_adaptive(f, a, b, nst = 1, tol = 1.e-8, max_iterations = 16):
    Iprev = 0.
    n = nst
    Iprev = trapezoidal_rule(f, a, b, n)
    print("Iteration: {0:5}, I = {1:20.15f}".format(1, Iprev))
    for k in range(1, max_iterations):
        n *= 2
        Inew = trapezoidal_rule(f, a, b, n)
        ek = (Inew - Iprev) / 3.
        print("Iteration: {0:5}, I = {1:20.15f}, error estimate = {2:10.15f}".format(k+1, Inew, ek))
        if (abs(ek) < tol):
            return Inew
        Iprev = Inew
    print("Failed to achieve tolerance")
    return Iprev
```

Computing the integral of $x^4 - 2x + 2$ over the interval (0.0 , 2.0) using adaptive trapezoidal rule

Iteration:	1, I =	16.000000000000000	
Iteration:	2, I =	9.000000000000000	error estimate = -2.333333333333333
Iteration:	3, I =	7.062500000000000	error estimate = -0.645833333333333
Iteration:	4, I =	6.566406250000000	error estimate = -0.165364583333333
Iteration:	5, I =	6.441650390625000	error estimate = -0.041585286458333
Iteration:	6, I =	6.410415649414062	error estimate = -0.010411580403646
Iteration:	7, I =	6.402604103088379	error estimate = -0.002603848775228
Iteration:	8, I =	6.400651037693024	error estimate = -0.000651021798452
Iteration:	9, I =	6.400162760168314	error estimate = -0.000162759174903
Iteration:	10, I =	6.400040690088645	error estimate = -0.000040690026556
Iteration:	11, I =	6.400010172525072	error estimate = -0.000010172521191
Iteration:	12, I =	6.400002543131352	error estimate = -0.000002543131240
Iteration:	13, I =	6.400000635782950	error estimate = -0.000000635782801
Iteration:	14, I =	6.400000158945742	error estimate = -0.000000158945736
Iteration:	15, I =	6.400000039736406	error estimate = -0.000000039736446
Iteration:	16, I =	6.400000009934106	error estimate = -0.000000009934100

Adaptive Simpson rule

For Simpson's rule $\varepsilon_k \simeq (I_S^k - I_S^{k-1})/15$ (understand why 15 and not 3?)

```
# Simpson's rule for numerical integration with adaptive step
def simpson_rule_adaptive(f, a, b, nst = 2, tol = 1.e-8, max_iterations = 16):
    Iprev = 0.
    n = nst
    Iprev = simpson_rule(f, a, b, n)
    print("Iteration: {0:5}, I = {1:20.15f}".format(1, Iprev))
    for k in range(1, max_iterations):
        n *= 2
        Inew = simpson_rule(f, a, b, n)
        ek = (Inew - Iprev) / 15.

        print("Iteration: {0:5}, I = {1:20.15f}, error estimate = {2:10.15f}".format(k+1, Inew, ek))
        if (abs(ek) < tol):
            return Inew
    Iprev = Inew
```

```
print("Failed to ac
return Inew

Computing the integral of x^4 - 2x + 2 over the interval ( 0.0 , 2.0 ) using adaptive Simpson's rule
Iteration:      1, I =      6.666666666666666
Iteration:      2, I =      6.416666666666666, error estimate = -0.016666666666667
Iteration:      3, I =      6.401041666666666, error estimate = -0.001041666666667
Iteration:      4, I =      6.400065104166666, error estimate = -0.000065104166667
Iteration:      5, I =      6.400004069010416, error estimate = -0.000004069010417
Iteration:      6, I =      6.400000254313150, error estimate = -0.000000254313151
Iteration:      7, I =      6.400000015894571, error estimate = -0.000000015894572
Iteration:      8, I =      6.400000000993410, error estimate = -0.000000000993411
```

Adaptive quadratures: Romberg method

Recall that we obtained error estimate for trapezoidal method at step k

$$\varepsilon_k \simeq (I_{\text{trap}}^k - I_{\text{trap}}^{k-1})/3$$

On the other hand, by definition, $\varepsilon_k = I - I_{\text{trap}}^k$

Therefore, we can improve our estimate of the integral as

$$I = R_{k,1} = I_{\text{trap}}^k + \frac{I_{\text{trap}}^k - I_{\text{trap}}^{k-1}}{3} + \mathcal{O}(h^4)$$

Romberg method: continue this procedure iteratively

$$R_{k,m+1} = R_{k,m} + \frac{R_{k,m} - R_{k-1,m}}{4^m - 1} .$$

until the desired accuracy is reached

Romberg method

```
def romberg(
    f,
    a,
    b,
    accuracy=1e-8,
    max_order=10
):
    R = np.zeros((max_order, max_order))
    h = (b - a) / 2.
    R[0, 0] = h * (f(a) + f(b)) # The initial trapezoidal rule
    for n in range(1, max_order):
        trapezoid = 0.0
        for j in range(2**(n-1)):
            trapezoid += f(a + (2*j+1)*h)
        R[n, 0] = 0.5 * R[n-1, 0] + h * trapezoid # The trapezoidal rule
        l = 1
        # The Romberg iterations
        for m in range(1, n+1):
            l *= 4
            R[n, m] = (l * R[n, m-1] - R[n-1, m-1]) / (l-1)
            print("Iteration: {0:5}, I = {1:20.15f}, error estimate = {2:10.15f}".format(n, R[n, m], abs(R[n, m] - R[n-1, m-1])))
            if abs(R[n, m] - R[n-1, m-1]) < accuracy:
                return R[n, m]
        h /= 2.
    print("Romberg method did not converge to required accuracy")
    return R[-1, -1]
```

Computing the integral of $x^4 - 2x + 2$ over the interval (0.0 , 2.0) using Romberg method

```
Iteration:    1, I =    6.666666666666667, error estimate = 9.333333333333332
Iteration:    2, I =    6.400000000000000, error estimate = 0.2666666666666667
Iteration:    3, I =    6.400000000000000, error estimate = 0.000000000000000
```

Improper integrals: Singularities at endpoints

- Even though if the singularities at integration endpoints are integrable, the trapezoidal, Simpson, etc. methods will fail because they evaluate the integrand at the endpoints

$$\int_0^1 \frac{1}{\sqrt{x}} dx = 2\sqrt{x} \Big|_0^1 = 2$$

```
def fsing1(x):  
    return 1./np.sqrt(x)  
  
trapezoidal_rule(fsing1,0.,1.,10)  
  
/tmp/ipykernel_31240/847063500.py:2: RuntimeWarning: divide by zero encountered in double_scalars  
    return 1./np.sqrt(x)
```

- **Solution:** use method that does use the endpoints (e.g. rectangle rule)

Improper integrals: Singularities at endpoints

$$\int_0^1 \frac{1}{\sqrt{x}} dx = 2\sqrt{x} \Big|_0^1 = 2$$

```
def fsing1(x):  
    return 1./np.sqrt(x)
```

```
print('Using rectangle rule to evaluate \int_0^1 1/\sqrt{x} dx')  
nst = 1  
rectangle_rule_adaptive(fsing1,0.,1.,1,1.e-3,20)
```

Using rectangle rule to evaluate \int_0^1 1/\sqrt{x} dx

Iteration:	1, I =	1.414213562373095	
Iteration:	2, I =	1.577350269189626,	error estimate = 0.054378902272177
Iteration:	3, I =	1.698844079579673,	error estimate = 0.040497936796682
Iteration:	4, I =	1.786461001734842,	error estimate = 0.029205640718390
Iteration:	5, I =	1.848856684639738,	error estimate = 0.020798560968299
Iteration:	6, I =	1.893088359706383,	error estimate = 0.014743891688882
Iteration:	7, I =	1.924392755699513,	error estimate = 0.010434798664376
Iteration:	8, I =	1.946535279970520,	error estimate = 0.007380841423669
Iteration:	9, I =	1.962194152677056,	error estimate = 0.005219624235512
Iteration:	10, I =	1.973267083679453,	error estimate = 0.003690977000799
Iteration:	11, I =	1.981096937261288,	error estimate = 0.002609951193945
Iteration:	12, I =	1.986633507070365,	error estimate = 0.001845523269692
Iteration:	13, I =	1.990548459938304,	error estimate = 0.001304984289313
Iteration:	14, I =	1.993316751362098,	error estimate = 0.000922763807931